TRAINING AND SCALING LANGUAGE MODELS: FROM FIRST PRINCIPLES TO EFFICIENT SERVING

Module 4: Single-GPU Performance

From a GPT Training Step to Faster GPU Execution

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. From a training step to GPU work
2. The machine beneath the tensor
3. Roofline reasoning
4. Memory, compilation and profiling

Guiding question

Is the GPU waiting for arithmetic, memory, kernels, or data, and what evidence distinguishes them?

What should be clear by the end?

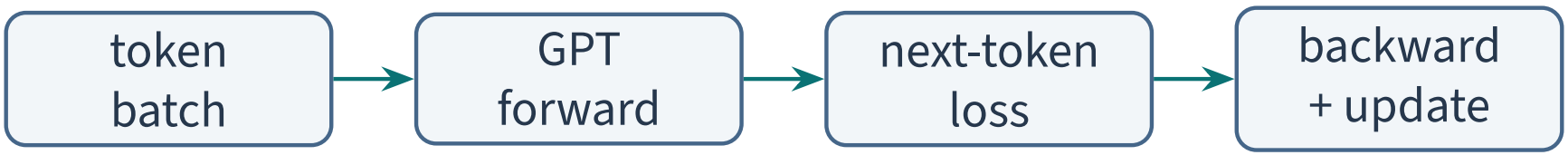
1. Connect the GPT training step to CPU dispatch and GPU kernels.
2. Use memory hierarchy and arithmetic intensity to reason about bottlenecks.
3. Explain kernel fusion and what `torch.compile` does.
4. Profile an update and measure changes without losing correctness.

PART 1

From a training step to GPU work

1. From a training step to GPU work
2. The machine beneath the tensor
3. Roofline reasoning
4. Memory, compilation and profiling

The workload we already built



Inside one Transformer block

Linear projections and MLP → matrix multiplications; attention → matrix products, masking and softmax; LayerNorm, GELU and residuals → reductions and elementwise work.

The same model and batch can run on CPU or GPU; compare the work and its execution, not just device names.

CPU and GPU: different ways to execute work

	CPU	GPU
Design	Fewer complex cores; strong per-thread latency	Many execution lanes; high throughput on parallel work
Good fit	Python, I/O, control flow; small or irregular work	Large batches of regular tensor operations
Memory	Host memory and caches	Device memory and on-chip caches
Trade-off	Less transfer/launch overhead for small work	Launch, transfer and synchronization costs

Measure the actual workload

A GPU can lose on small shapes or transfer-heavy work. Report batch, dtype, threads, device, and what is timed.

What happens when PyTorch runs on CUDA?

1. CPU prepares a batch and issues PyTorch operations.
2. Tensors are copied to the device when needed; GPU-resident weights and activations stay in device memory.
3. The CPU enqueues GPU kernels; the GPU executes them asynchronously.
4. Backward launches more kernels; the optimizer updates weights.

Timing trap

A Python timer around an asynchronous launch can stop before the GPU finishes. Synchronize at timing boundaries or use CUDA events.

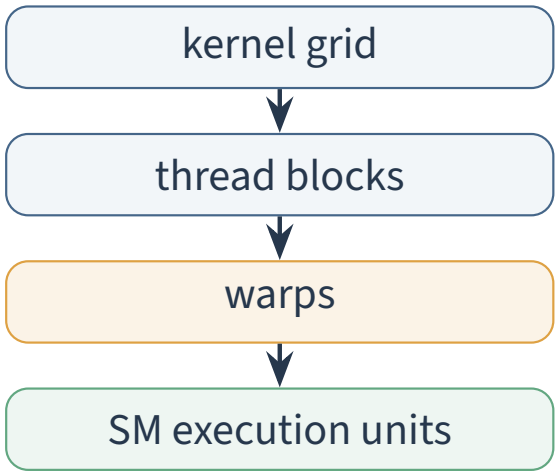
PART 2

The machine beneath the tensor

1. From a training step to GPU work
2. The machine beneath the tensor
3. Roofline reasoning
4. Memory, compilation and profiling

A GPU is a throughput machine with many lanes

- A kernel launches a grid of threads.
- Threads execute in warps, commonly 32 lanes.
- Warps are scheduled onto streaming multiprocessors (SMs).
- Tensor cores accelerate supported matrix tiles.



Source: JAX Scaling Book, How to Think About GPUs

Warps reward regular work

Good utilization

contiguous access
similar control flow
enough active warps
tile-aligned matrix dimensions

Lost utilization

branch divergence
strided loads
register pressure
tiny or awkward shapes

A mathematically identical operation can run much faster when its layout and dimensions match the hardware tiles.

The memory hierarchy trades capacity for speed

Level	Scope	Capacity	Role
registers	one thread	tiny	live scalar and fragment state
shared memory / SRAM	one block or SM	small	explicit reuse and tiling
L2 cache	whole device	moderate	cross-SM reuse
HBM / VRAM	whole device	large	weights, activations, optimizer state
host memory	CPU system	largest	dataset staging and overflow

Performance principle

Moving a value often costs more time and energy than performing one arithmetic operation on it. Reuse data near the compute units.

Map GPT operations to hardware questions

In our training step	Typical work	Ask in the trace
QKV and MLP	Matrix multiplications	Are shapes large enough?
Attention	Products and softmax	Which backend ran?
Norm, GELU, residual	Reductions and pointwise ops	Are launches or intermediates costly?
Data, loss, optimizer	Copies and updates	Is the GPU waiting?

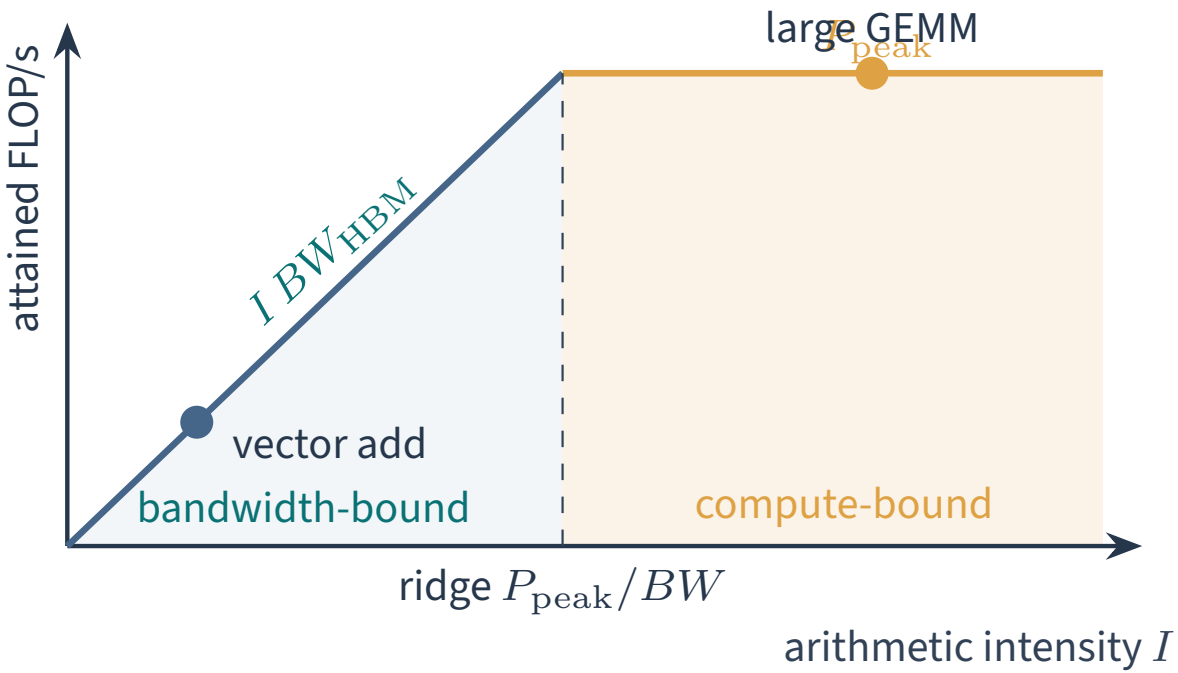
Kernel choices depend on shape, dtype, hardware and implementation. Measure before drawing a conclusion.

PART 3

Roofline reasoning

1. From a training step to GPU work
2. The machine beneath the tensor
3. Roofline reasoning
4. Memory, compilation and profiling

Arithmetic intensity predicts the limiting resource



$$I = \frac{F}{B_{\text{HBM}}}$$

$$P \leq \min(P_{\text{peak}}, I\ BW)$$

Read the lower roof

Moving right means reusing each loaded byte more.

Source: JAX Scaling Book, How to Think About GPUs

Vector addition: a bandwidth-bound model

For $C = A + B$ with FP32 elements:

$$\text{FLOPs} = 1, \qquad \text{bytes} = 4 + 4 + 4 = 12$$

$$I \approx 0.083 \text{ FLOP/byte}$$

Prediction

For a large vector under this traffic model, bandwidth can dominate. Small vectors can instead be launch-limited.

Benchmark check

Doubling peak FLOPs should barely change runtime if memory bandwidth stays fixed.

Large matrix multiplication creates reuse

For square $C = AB$, matrices of width n:

$$\text{FLOPs} \approx 2n^3, \qquad \text{minimum bytes} \approx 3n^2 s$$

$$I \approx \frac{2n}{3s}$$

This idealized intensity assumes each matrix element is transferred once. Tiling increases reuse; actual traffic may be higher.

Small matrices behave differently

Launch overhead, poor tile occupancy, and insufficient reuse can keep small GEMMs far below peak compute.

Fusion: the same math, fewer intermediates

Consider an illustrative pointwise chain $y = \mathrm{ReLU}(x + b) + r$, with equal-sized tensors:

Separate kernels

$u = x + b \quad \rightarrow$ write u

$v = \mathrm{ReLU}(u) \quad \rightarrow$ write v

$y = v + r \quad \rightarrow$ write y

One fused kernel

Read x, b, r; compute the chain; write y.

u and v can stay in registers instead of being written and re-read.

Idealized traffic: 8 versus 4 element transfers. Cache reuse, broadcasting and actual kernels change the result; measure the trace.

FlashAttention changes traffic, not the attention equation

Naive attention materializes the $T \times T$ score and probability matrices in HBM. FlashAttention tiles the computation and maintains online softmax statistics in SRAM.

Naive path

write scores
read for softmax
write probabilities
read to multiply values

Tiled path

load Q, K, V blocks
update exact softmax statistics
write only the final output

Checkpoint: roofline diagnosis

Think, then compare

An optimization halves the FLOP count of a normalization kernel but runtime barely moves. Which roofline quantity explains the result, and what change is more promising?

PART 4

Memory, compilation and profiling

---

1. From a training step to GPU work
2. The machine beneath the tensor
3. Roofline reasoning
4. Memory, compilation and profiling

VRAM holds much more than model weights

Component	Typical scaling	Lifetime
parameters	$P\times$ dtype bytes	whole run
gradients	roughly P	until update/reset
Adam moments	roughly $2P$ in FP32	whole run
master weights	optional FP32 copy	whole run
activations	B, T, L, d dependent	forward to backward
temporary buffers	kernel dependent	operation lifetime
allocator reserve	fragmentation dependent	process lifetime

Useful distinction

Report allocated and reserved memory. Out-of-memory behavior depends on peaks and fragmentation, not only steady-state tensor sizes.

Activation checkpointing exchanges compute for memory

Store everything

Faster backward pass
higher activation memory
larger feasible recomputation-free batch

Checkpoint selected boundaries

Lower activation memory
recompute forward segments in backward
may enable larger B or T

Evaluate end-to-end tokens per second. A larger batch does not guarantee a faster run if recomputation dominates.

Performance metrics answer different questions

Throughput

tokens/s measures delivered training work
step time exposes update cadence
time-to-target includes optimization quality

Hardware efficiency

$$\mathrm{MFU} = \frac{\text{model FLOPs/s}}{\text{peak FLOPs/s}}$$

MFU depends on the FLOP model and quoted peak precision.

What does `torch.compile` do?

1. Wrap a model or function: `compiled = torch.compile(model)`.
2. On execution, TorchDynamo captures supported PyTorch operations in graphs with guards on assumptions such as shapes.
3. For training, PyTorch captures forward and backward regions. The default TorchInductor backend generates optimized code, potentially fusing operations.
4. Later calls reuse compiled code when the guards hold.

Goal, not guarantee

Reduce Python/launch overhead and memory traffic, and choose efficient kernels. Some operations remain library calls; measure the whole step.

Source: PyTorch torch.compile documentation

Compilation: costs, breaks and correctness

Why might it not help?

First calls include compilation. Graph breaks split regions; changing shapes can trigger recompilation. Some workloads are already dominated by optimized library kernels.

How to evaluate

Compare eager and compiled with the same model, input and precision. Warm up; synchronize; check loss and gradients within tolerances; inspect the trace.

`torch.compile` and efficient SDPA are complementary: SDPA selects an attention implementation; compilation optimizes captured regions around it.

Read one training update on a timeline

CPU lane	prepare / launch kernels	next work
GPU lane	copy (if needed) $\rightarrow$ forward $\rightarrow$ backward $\rightarrow$ optimizer	

- Zoom from the whole update to a long gap or expensive kernel; link it back to attention, MLP, normalization, loss or optimizer.
- Many short kernels suggest a fusion or launch-overhead hypothesis; confirm by comparing traces and timings.
- A copy or CPU gap may matter end to end even if model-only device timings look fast.

The lab uses fixed synthetic batches already on the device; it does not time the data loader or host-to-device copies.

Read a profiler trace as a causal story

1. Is the GPU idle between steps? Inspect input loading and synchronization.
2. Are there many tiny kernels? Inspect fusion and compilation opportunities.
3. Do copies overlap compute? Inspect pinned memory and asynchronous transfer.
4. Does one kernel dominate? Classify it with shapes, bytes, and FLOPs.
5. Are unexpected synchronizations present? Find scalar reads and host barriers.

Apply optimizations in an evidence-driven order

1. verify correctness and stabilize the benchmark
2. keep the input pipeline ahead of the GPU
3. test a supported lower precision with numerical checks
4. compare manual attention with SDPA
5. investigate unfused pointwise chains
6. compare eager with `torch.compile`
7. checkpoint activations only when memory is the constraint

One change per measurement

Warm up, synchronize, use enough repeated steps, and report median plus dispersion.

Checkpoint: one-change ablation

Think, then compare

Choose one slow trace pattern. State the bottleneck hypothesis, the single intervention, the expected profiler change, and the correctness check that must remain unchanged.

Key takeaways

1. The course GPT update becomes CPU dispatch, GPU kernels and memory traffic.
2. Roofline reasoning separates bandwidth-bound from compute-bound operations.
3. Fusion can reduce intermediate traffic; `torch.compile` may generate fused kernels.
4. A trustworthy speedup needs stable shapes, warmup, synchronization, correctness checks and profiling evidence.